

CSAPP 考前总结

刘鼎琨

2023.6.7

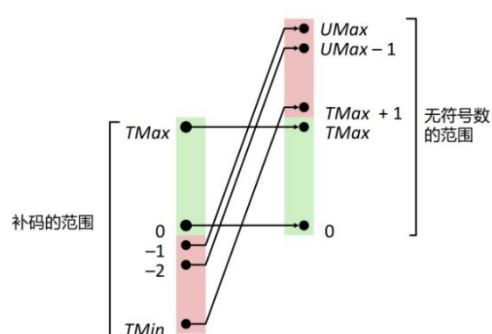
复习材料

讲义、课堂练习、课后作业和三次实验

第2章

整数、浮点数及相关运算的性质(无符号/有符号转换、溢出、精度损失等);

1. 无符号/有符号转换



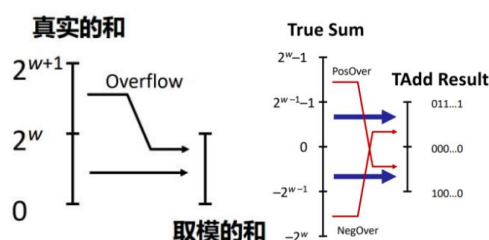
考虑拓展和截断.

相关练习题在讲义上.

简单来说, 相同的位级表示, 但解读不同!

2. 溢出:

[加法与乘法]



```
int uadd_ok(unsigned x, unsigned y)
{
    unsigned sum = x+y;
    return sum >= x;
}
```

```

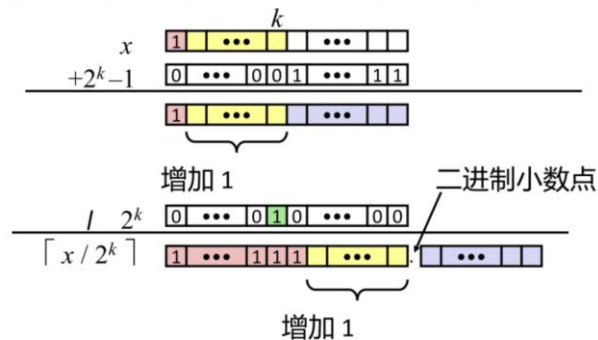
int tadd_ok(int x, int y)
{
    int sum = x+y;
    int neg_over = x < 0 && y < 0 && sum >= 0;
    int pos_over = x >= 0 && y >= 0 && sum < 0;
    return !neg_over && !pos_over;
}

int tmult_ok(int x, int y)
{
    int p = x*y;
    return !x || p/x == y;
}

```

[除法]

- 带移位的**无符号数** 2 次幂除法——逻辑移位
- 带移位的**有符号** 2 次幂除法
若采用算术移位，则向下舍入。
若希望向 0 舍入，采用有偏置的二次幂除法。 $(x + (1 \ll k) - 1) \gg k$



注：只要打算右移舍去的位中有一个高位，势必发生进位！

[浮点数]

【基本思想】

- 首先计算**精确**结果；
- 使之**适配**到期望的精度；
- 如果**指数**太大，可能会溢出；
- 可能需要**舍入**来适配**小数 frac** 字段 —— **向偶数舍入法**

$$1.010 \cdot 2^2 + 1.110 \cdot 2^3 = (0.1010 + 1.1100) \cdot 2^3 = 10.0110 \cdot 2^3 = 1.00110 \cdot 2^4 = 1.010 \cdot 2^4$$

注：如果 E 超出范围，则溢出

Format A		Format B		Comments
Bits	Value	Bits	Value	
1 01111 001	$-\frac{9}{8}$	1 0111 0010	$-\frac{9}{8}$	
0 10110 011	176	0 1110 0110	176	
1 00111 010	$-\frac{5}{1024}$	1 0000 0101	$-\frac{5}{1024}$	Norm $\rightarrow$ denorm
0 00000 111	$\frac{7}{131072}$	0 0000 0001	$\frac{1}{1024}$	Smallest positive denorm
1 11100 000	-8192	1 1110 1111	-248	Smallest number $> -\infty$
0 10111 100	384	0 1111 0000	$+\infty$	Round to ∞ .

3. 精度损失:

- int, float, 和 double 之间的转换 **会改变位表示**
- double/float $\rightarrow$ int 截断小数部分, 向零舍入, 超出表示范围或 NaN 时, 行为未定义: 通常设置为 T_{Min}
- int $\rightarrow$ double 精确转换, 只要 int ≤ 53 位
- int $\rightarrow$ float 可能会进行舍入

4. 运算性质判定:

- A. $(x < y) == (-x > -y)$. No, Let $x = TMin_{32}$, $y = 0$.
- B. $((x+y) << 4) + y - x == 17*y + 15*x$. Yes, from the ring properties of two's complement arithmetic.
- C. $\sim x + \sim y + 1 == \sim(x+y)$. Yes, $\sim x + \sim y + 1 = (-x-1) + (-y-1) + 1 = -(x+y) - 1 = \sim(x+y)$.
- D. $(ux - uy) == -(\text{unsigned})(y-x)$. Yes. Due to the isomorphism between two's complement and unsigned arithmetic.
- E. $((x \gg 2) << 2) \leq x$. Yes. Right shift rounds toward minus infinity.
- A. $(\text{float}) x == (\text{float}) dx$. Yes. Converting to float could cause rounding, but both x and dx will be rounded in the same way.
- B. $dx - dy == (\text{double})(x-y)$. No. Let $x = 0$ and $y = TMin_{32}$.
- C. $(dx + dy) + dz == dx + (dy + dz)$. Yes. Since each value ranges between $TMin_{32}$ and $TMax_{32}$, their sum can be represented exactly.
- D. $(dx * dy) * dz == dx * (dy * dz)$. No. Let $dx = TMax_{32}$, $dy = TMax_{32} - 1$, $dz = TMax_{32} - 2$. (Not detected with Linux/gcc)
- E. $dx / dx == dz / dz$. No. Let $x = 0$, $z = 1$.

注: 0, Tmin, Tmax 都是反例.

5. 如果考察 datalab, 基本设计思路:

1. 在解决限制空间和运算符的问题时, 摒弃使用循环的想法至关重要!
2. 许多有用的结构确实有帮助! 像 $(x \gg 31) + 1$; 简单地说就是取 x 的高位电平, 它可以很快. 同时, 像掩码的方式也很常见.
3. 编码前最好有一个具体的计划.
4. 在设计程序时, 要注意可能导致大问题的小情况.

浮点数的计算；

【基本思想】

- 首先计算**精确**结果；
- 使之**适配**到期望的精度；
- 如果**指数**太大，可能会溢出；
- 可能需要舍入来适配**小数 frac** 字段 —— **向偶数舍入法**

$$1.010 \cdot 2^2 + 1.110 \cdot 2^3 = (0.1010 + 1.1100) \cdot 2^3 = 10.0110 \cdot 2^3 = 1.00110 \cdot 2^4 = 1.010 \cdot 2^4$$

注：如果 E 超出范围，则溢出

- 加法运算律：
 - 满足交换律、单调性（除了无穷大和 NaN）
 - 不满足结合律：如 $(3.14 + 1e20) - 1e20 = 0$, $3.14 + (1e20 - 1e20) = 3.14$
- 乘法运算律：
 - 满足交换律、单调性（除了无穷大和 NaN）
 - 不满足结合律：如 $(1e20 \cdot 1e20) \cdot 1e-20 = \text{inf}$, $1e20 \cdot (1e20 \cdot 1e-20) = 1e20$
 $1e20 \cdot (1e20 - 1e20) = 0.0$, $1e20 \cdot 1e20 - 1e20 \cdot 1e20 = \text{NaN}$

浮点数的构成；

描述	Hex	M	E	V	D
-0	0x8000	0	-14	-0-0	-0.0
最小的>2的值	0x4001	1025/1024	1	1025/512	2.00195312
512	0x6000	1	9	512	512.0
最大的非规格化数	0x03FF	1023/1024	-14	$1023/(2^{24})$	$6.09755516e-5$
$-\infty$	0xFC00	-	-	$-\infty$	$-\infty$
十六进制表示为38B0的数	0x3BB0	123/64	-1	123/128	0.9609375

```
1 /* Compute 2**x */
2 float fpwr2(int x) {
3
4     unsigned exp, frac;
5     unsigned u;
6
7     if (x < -149) {
8         /* Too small. Return 0.0 */
9         exp = 0;
10        frac = 0;
11    } else if (x < -126) {
12        /* Denormalized result */
13        exp = 0;
14        frac = 1 << (x + 149);
15    } else if (x < 128) {
16        /* Normalized result. */
17        exp = x + 127;
18        frac = 0;
19    } else {
20        /* Too big. Return +oo */
21        exp = 255;
22        frac = 0;
23    }
24    u = exp << 23 | frac;
25    return u2f(u);
26 }
```

第3章

基本指令及寻址方式计算；

cltq(cqo) %rax 符号拓展给%rdx. 为 64 位除法做准备.
cld %eax 符号拓展给%edx. 为 32 位除法做准备.
cltq %eax 符号拓展给%rax.

imulq	S	$R[\%rdx]: R[\%rax] \leftarrow S \times R[\%rax]$	有符号全乘法
mulq	S	$R[\%rdx]: R[\%rax] \leftarrow S \times R[\%rax]$	无符号全乘法
idivq	S	$R[\%rdx] \leftarrow R[\%rdx]: R[\%rax] \bmod S$ $R[\%rax] \leftarrow R[\%rdx]: R[\%rax] \div S$	有符号除法
divq	S	$R[\%rdx] \leftarrow R[\%rdx]: R[\%rax] \bmod S$ $R[\%rax] \leftarrow R[\%rdx]: R[\%rax] \div S$	无符号除法

leaq 指令是唯一一个只寻址，不取址的指令.

%fs: 40 就是在 40 这个位置选取出要求长度的值，简单来说就是获得一段规定长度的随机序列.

movl 特殊，高 32 位零扩展. 32 位的指令会零扩展到高 32 位.

读写 4 个条件码的相关指令，跳转指令的跳转条件，分支/循环/switch 语句及跳转表分析；

设置CF 如果最高有效位产生进位/借位（无符号溢出）

设置ZF 如果 $t == 0$

设置SF 如果 $t < 0$ (用于有符号数)

设置OF 如果补码溢出（有符号，正溢出或负溢出）

$(a > 0 \ \&\& \ b > 0 \ \&\& \ t < 0) \ || \ (a < 0 \ \&\& \ b < 0 \ \&\& \ t \geq 0)$

(2) 隐式设置:

- 常用算数与逻辑指令集中, 除却 leaq 之外, 都会设置.
- 比较特殊: xor 进位和溢出的标志置 0. 移位操作, 进位为最后被移出的位, 溢出标志置 0.

(3) 显式设置:

指令	基于	描述
CMP S_1, S_2	$S_2 - S_1$	比较
cmpb		比较字节
cmpw		比较字
cmpl		比较双字
cmpq		比较四字
TEST S_1, S_2	$S_1 \& S_2$	测试
testb		测试字节
testw		测试字
testl		测试双字
testq		测试四字

注: testq %rax, %rax 可以实现检查%rax 是否为正数, 负数和零.

(4) 显示读取:

指令	同义名	效果	设置条件
sete D	setz	$D \leftarrow ZF$	相等/零
setne D	setnz	$D \leftarrow \sim ZF$	不等/非零
sets D		$D \leftarrow SF$	负数
setns D		$D \leftarrow \sim SF$	非负数
setg D	setnle	$D \leftarrow \sim (SF \wedge OF) \& \sim ZF$	大于 (有符号 >)
setge D	setnl	$D \leftarrow \sim (SF \wedge OF)$	大于或等于 (有符号 >=)
setl D	setnge	$D \leftarrow SF \wedge OF$	小于 (有符号 <)
setle D	setng	$D \leftarrow (SF \wedge OF) \vee ZF$	小于或等于 (有符号 <=)
seta D	setnbe	$D \leftarrow \sim CF \& \sim ZF$	超过 (无符号 >)
setae D	setnb	$D \leftarrow \sim CF$	超过或相等 (无符号 >=)
setb D	setnae	$D \leftarrow CF$	低于 (无符号 <)
setbe D	setna	$D \leftarrow CF \vee ZF$	低于或相等 (无符号 <=)

注: i. 分清楚有符号和无符号的指令差异.

ii. set 和 cmp 共同使用时, 逻辑顺序是颠倒的, 需要反过来看.

iii. 基本操作流程: D 为 1 个字节大小, 可能需要扩展到 64 位.

(5) 跳转指令:

指令	同义名	跳转条件	描述
jmp $Label$		1	直接跳转
jmp $*Operand$		1	间接跳转
jz $Label$	jz	ZF	相等/零
jne $Label$	jnz	$\sim ZF$	不相等/非零
js $Label$		SF	负数
jns $Label$		$\sim SF$	非负数
jg $Label$	jnle	$\sim (SF \wedge OF) \& \sim ZF$	大于 (有符号 >)
jge $Label$	jnl	$\sim (SF \wedge OF)$	大于或等于 (有符号 >=)
jle $Label$	jnge	$SF \wedge OF$	小于 (有符号 <)
jle $Label$	jng	$(SF \wedge OF) \vee ZF$	小于或等于 (有符号 <=)
ja $Label$	jnbe	$\sim CF \& \sim ZF$	超过 (无符号 >)
jae $Label$	jnb	$\sim CF$	超过或相等 (无符号 >=)
jb $Label$	jnae	CF	低于 (无符号 <)
jbe $Label$	jna	$CF \vee ZF$	低于或相等 (无符号 <=)

注: jmp %rax 和 jmp *(%rax) 代表跳转到相应地址的绝对跳转.

关于循环:

一般情况下都在循环之前有一次判定, 本质都是 while 类型的.

关于 Switch:

使用情况是当开关数量比较多, 范围跨度小的情况.

有不同的跳转表形式:

```

func:
    cmpl $6, %r8d
    ja    .L2
    subq  $8, %rsp
    movq  %rdx, %r9
    movl  %r8d, %r8d
    leaq  .L4(%rip), %rdx
    movslq (%rdx,%r8,4), %rax
    addq  %rdx, %rax
    jmp   *%rax

.L4:
    .long .L3-.L4
    .long .L5-.L4
    .long .L6-.L4
    .long .L7-.L4
    .long .L8-.L4
    .long .L7-.L4
    .long .L9-.L4

switcher:
    leal  -17(%rdi), %eax
    cmpl  $6, %eax
    ja    .L8
    movl  %eax, %eax
    jmp   *.L4(,%rax,8)

.L4:
    .quad .L7
    .quad .L6
    .quad .L5
    .quad .L8
    .quad .L5
    .quad .L8
    .quad .L3

```

此处, quad 为 8 字节, long 为 4 字节.

过程调用中栈的状态分析, 控制转移、数据传送、内存管理机制, 递归函数调用的实现;

栈的状态分析

0x7fffffff7a8	调用series(3)的返回地址
0x7fffffff7a0	%rbp的值, 未知
0x7fffffff798	%rbx的值, 未知
0x7fffffff790	未知
0x7fffffff788 rsp→	调用series(1)的返回地址 0x55555554759

控制转移

(3) 转移控制: 对应上图“返回地址”部分.

指令	描述
call <i>Label</i>	过程调用
call <i>*Operand</i>	过程调用
ret	从过程调用中返回

注: i. call 命令会将返回地址(当前命令地址的下一命令)压栈, 同时令%rip(PC)为 label 的地址. Ret 命令则弹出返回地址, 赋值给%rip(PC), 由此分析可知无需关注具体的压栈和出栈逻辑.

ii. 函数调用时尤其在支持变长的栈帧中可以选用%rbp, 稳定性强.

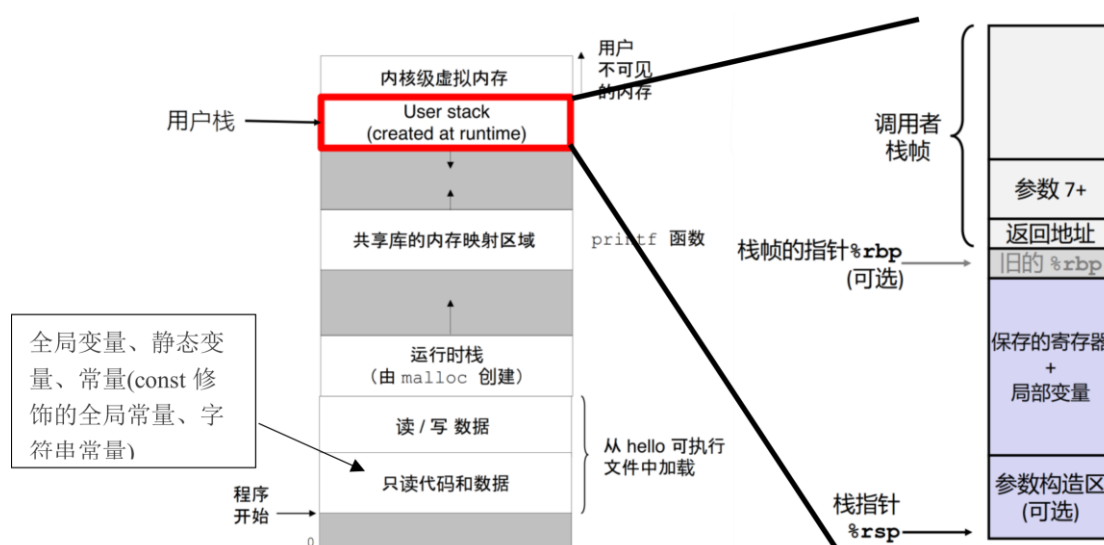
数据传送

操作数大小(位)	参数数量					
	1	2	3	4	5	6
64	%rdi	%rsi	%rdx	%rcx	%r8	%r9
32	%edi	%esi	%edx	%ecx	%r8d	%r9d
16	%di	%si	%dx	%cx	%r8w	%r9w
8	%dil	%sil	%dl	%cl	%r8b	%r9b

注: i. 如果传递的参数是 `int` 类型的, 则高 32 位为未定义, 不能也不应该调用.

ii. 如果传递的参数多余六个, 则需要内存压栈实现, 其中参数 7 位于栈顶, 所有数据向 8 的倍数对齐.

内存管理机制



(5) 局部存储: 对应上图“局部变量”部分.

• 什么情况下使用局部内存: 寄存器不足以放置所有本地数据, 使用“&”必须为之分配地址, 局部变量是数组、结构.

(6) 局部存储空间: 对应上图“保存的寄存器”部分.



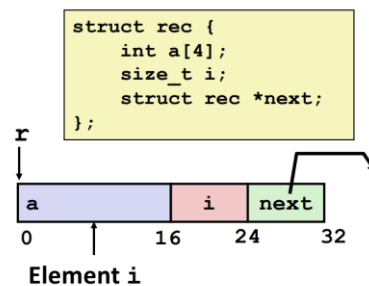
递归函数调用的实现;

递归问题, 一般先判定跳出条件, 比较容易. 之后需要在递归之前保存好相关变量. 递归之后再释放出来.

遍历链表#2

■ C 代码

```
void set_val
(struct rec *r, int val)
{
    while (r) {
        size_t i = r->i;
        // No bounds check
        r->a[i] = val;
        r = r->next;
    }
}
```



Register	Value
%rdi	r
%rsi	val

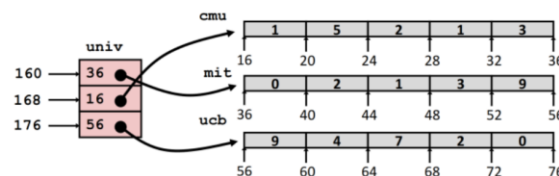
```
.L11:                                # loop:
movq 16(%rdi), %rax                 # i = Mem[r+16]
movl %esi, (%rdi,%rax,4)            # Mem[r+4*i] = val
movq 24(%rdi), %rdi                 # r = Mem[r+24]
testq %rdi, %rdi                    # Test r
jne .L11                             # if !=0 goto loop
```

汇编代码的阅读填空题:

先大致判定考点, 如递归, 跳转表等, 再关注参数传递, 命令针对的数据大小, 预估可能的整体功能.

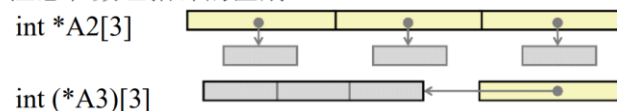
数组(一维、多维)的内存分配及访问, 结构体的分配(对齐)和访问;

- ① 一维数组: $&D[i] = x_D + L * I$;
- ② 二维数组: 若为“行优先”模式, 则: $&D[i][j] = x_D + L * (C * i + j)$;
- ③ 指针数组:



需要寻址两次: $\text{Mem}[\text{Mem}[\text{univ} + 8 * \text{index}] + 4 * \text{digit}]$

注: 注意和数组指针的区别



(2) 结构体:

■ 结构对齐的原则:

- 基本数据类型需要 K 字节表示, 则地址必须能是 K 的倍数
- 最大的对齐要求 K, 整体结构必须是 K 的倍数

■ 结构对齐的空间节省: 把大型数据类型放在首位

注: 元素是结构体的数组, 则整体结构长度乘以 K, 满足每个元素的对齐要求. 结构体中包含结构体的情形类似.

缓冲区溢出的原理及防范措施;

1. 原理

当超过为数组分配的内存大小时,导致的错误. 最常见的形式: 未检查输入字符串的长度, 特别是对于栈上有界的字符数组.

2. 处理缓冲区溢出攻击的几种方法:

- ① 改用更加安全的函数进行调用 避免代码中的溢出漏洞
- ② 栈随机化, 系统级保护
- ③ 不可执行的代码段, 系统级保护
- ④ 栈金丝雀

第 7 章

链接过程;

步骤 1: 符号解析.

将每个模块中引用的符号与某个目标模块中的定义符号建立关联.

步骤 2: 重定位

基于符号解析, 识别出有关联的目标模块

将这些模块中的节进行合并, 并确定运行时符号的地址

在引用符号的位置, 重定位引用地址

Elf 文件的基本结构;

(2) ELF 文件分区:

- ① ELF header: 魔数 ELF(7F 45 4C 46)、字长、字节序、文件类型等.
- ② 程序头部表: 页大小、虚拟地址内存段 (节, sections)、段大小等, 考虑如何映射, 可执行文件和动态链接库都有此版块.
- ③ .text 节: 代码节, 可读/可执行.
- ④ .rodata 节: 只读数据, 如跳转表.
- ⑤ .data 节: 初始化的全局变量, 可读/可写.
- ⑥ .bss 节: 未初始化的静态变量, 或初始化为 0 的全局和静态变量. 可读/可写. 具有节头部, 但是实际不占用空间.
注: 区分 COMMON 区, 隐藏在 ELF 文件中, 保存了未初始化的全局变量.
- ⑦ .symtab 节: 符号表.
- ⑧ .rel.text 节、.rel.data 节: 包含相关重定位信息.
- ⑨ .debug 节: 有关符号调试的信息.
- ⑩ 节头部表: 每个节的偏移量和大小.

ELF header
Program header table (required for executables)
.text section
.rodata section
.data section
.bss section
.symtab section
.rel.txt section
.rel.data section
.debug section
Section header table

符号的类别;

全局符号, 局部符号, 一定要区分于局部变量.

符号表及符号的解析过程;

符号表是一个 structs 的数组 § 每个条目包括符号的名字、大小和位置

- 符号表 (symtab) 中每个条目的结构如下 (64位系统)

```
typedef struct {
    int name; /*指向符号对应字符串在strtab节中的偏移*/
    char type: 4, /*符号对应目标的类型: 数据、函数、源文件、节*/
        binding: 4; /*符号对应目标是全局符号还是局部符号*/
    char reserved;
    short section; /*符号对应目标所在的section, 或其他情况*/
    int value; /*在对应section中的偏移量, 可执行文件中是虚拟地址*/
    int size; /*符号对应目标所占字节数*/
} Elf64_Symbol;
```

Symbol	swap.o .symtab entry?	Symbol type	Module where defined	Section
buf	yes	extern	m.o	.data
bufp0	yes	global	swap.o	.data
bufp1	yes	local	swap.o	.bss
swap	yes	global	swap.o	.text
temp	no	—	—	—
incr	yes	local	swap.o	.text
count	yes	local	swap.o	.bss

注：一定要审题！

- (4) 解读规则：

- 规则 1：不允许有多个同名的强符号。
- 规则 2：如果有一个强符号和多个弱符号，选择强符号。
 - 注：可能会向上覆盖，上方的变量无论顺序都会被覆盖。
- 规则 3：如果有多个弱符号，任意选择一个。

- (5) 编程建议：尽量避免全局变量；尽量使用 `static`；尽量初始化好全局变量；用好 `extern` 等标志。

重定位表及符号的重定位过程；

- 重定位结构体信息如下：offset、type、symbol、addend.

offset: 节偏移
 type: R_386_PC32 PC 相对寻址(跳转表等); R_386_32 绝对寻址(固定内存).
 symbol: 符号表示.
 addend: 常数偏移量(一般为负)

```
00000000004004d0 <main>:
4004d0: 48 83 ec 08      sub    $0x8,%rsp
4004d4: be 02 00 00 00   mov    $0x2,%esi
4004d9: bf 18 10 60 00   mov    $0x601018,%edi # %edi = &array
4004de: e8 05 00 00 00   callq 4004e8 <sum> # sum()
4004e3: 48 83 c4 08      add    $0x8,%rsp
4004e7: c3              retq

00000000004004e8 <sum>:
4004e8: b8 00 00 00 00   mov    $0x0,%eax

Relocation section '.rela.text' at offset 0x1e8 contains 2 entries:
Offset      Info          Type          Sym. Value    Sym. Name + Addend
000000000000a 000900000000a R_X86_64_32    0000000000000000 array + 0
000000000000f 000a000000002 R_X86_64_PC32  0000000000000000 sum - 4
```

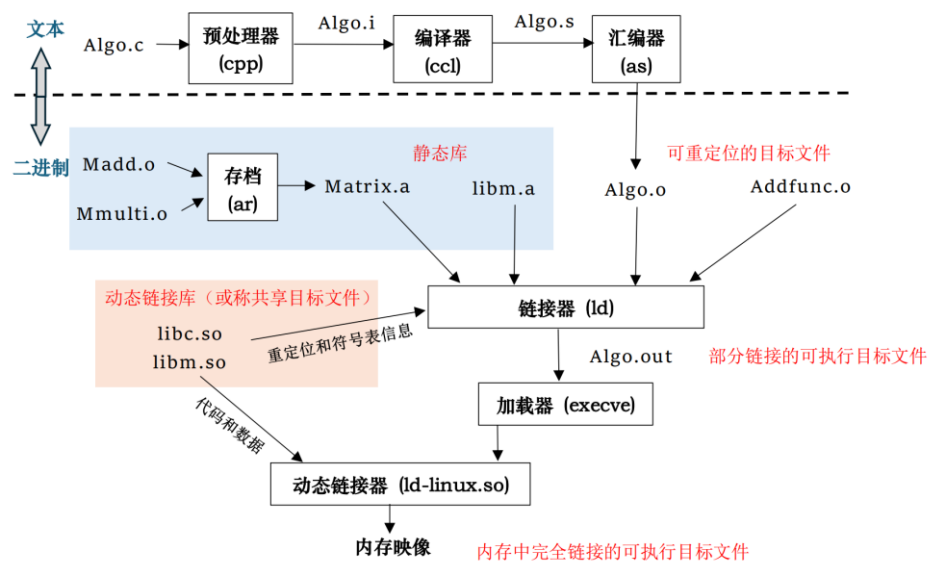
- 相对寻址的计算方法：

- ① $\text{refAddr} = \text{main.address} + \text{rela.offset} = 0x4004d0 + 0xf = 0x4004df$
- ② 执行 `callq`, $\text{PC} = \text{refAddr} - \text{rela.addend} = 0x4004df - (-4) = 0x4004e3$
- ③ 重定位: $\text{sum.addr} - \text{PC} = 0x4004e8 - 0x4004e3 = 0x5$
- ④ 按照小端模式进行填写。 注：PIC 为动态寻址的方式，常用。

静态库及其解析过程；

八、 静态库

- (1) 定义：将一些相关联的可重定位目标文件连接成一个带索引的文件（称为存档文件 archive）
- (2) 命令行链接的顺序很重要：链接器按命令行顺序扫描 .o 文件和 .a 文件
 - 原则：将库放在命令行的最后、被引用者放置在引用者后面。
- (3) 劣势：
 - 存储的可执行文件中有大量重复（每个函数都需要 libc）
注：关注的是代码和数据在磁盘存储上的冗余。
 - 运行的可执行文件中有大量重复
注：关注的是代码和数据在运行时内存中的冗余。
 - 对系统库的很小的 bug 修复，要求每个应用程序都进行重新链接



`gcc -static -o myfunc func.o libx.a libz.a liby.a`

举例：

- A. `p.o → libx.a → p.o`
`gcc p.o libx.a`
- B. `p.o → libx.a → liby.a` 和 `liby.a → libx.a`
`gcc p.o libx.a liby.a libx.a`
- C. `p.o → libx.a → liby.a → libz.a` 和 `liby.a → libx.a → libz.a`
`gcc p.o libx.a liby.a libx.a libz.a`

动态库的基本概念；

包含代码和数据的目标文件，在加载时或运行时，动态地加载和链接到应用程序中

九、 动态库（共享目标文件）

- 可以在可执行文件初次加载和运行时进行动态链接 (load-time linking)
 - ✧ Linux 上最常见的情况，由动态链接器 (ld-linux.so) 自动处理
 - ✧ 标准 C 库 (libc.so) 通常是动态链接的
- 可以在程序开始执行后进行动态链接 (run-time linking)
 - ✧ 在 Linux 上，是通过调用 dlopen() 接口来实现的

```
gcc -shared -o libvector.so addvec.c multvec.c
```

常见的插桩方法.

十、 库打桩/插桩

- (1) 库打桩定义：一种强大的链接技术，使得程序员可以截获对任意函数的调用。
- (2) 编译时打桩：需要.c 文件.
 - ✧ #define malloc(size) mymalloc(size)
 - ✧ #define free(ptr) myfree(ptr)将对 malloc/free 的调用宏扩展为对 mymalloc/myfree 的调用
- (3) 链接时打桩：需要.o 文件.
 - ✧ gcc -Wall -Wl,--wrap,malloc -Wl,--wrap,free -o intl
 - 使用链接器的技巧做特殊的名字解析，可以在 __wrap_free 内部使用 __real_free 来调用真正的 free 函数。
 - 相当于：Malloc → __wrap_malloc
 __real_malloc → malloc
- (4) 加载/运行时打桩：需要.out 文件.

利用 LD_PRELOAD 环境变量告诉动态链接器在解析未被解析的引用时 (例如，对 malloc 的引用)，先在自定义的 mymalloc.so 中查找。